SIGHT.SERIALIZATION · SPECIFICATION

SBDL Binary Format Specification

FORMAT

Sight Bundle, binary encoding (`.sbdl`)

FORMAT VERSION

1

DATE

2026-10-20

COMPANION DOCUMENT

SBDL Text Format Specification

CONTENTS

1. **Introduction**
   - 1.1 Conventions
   - 1.2 Notation used in layouts
2. **Overview**
3. **Data model**
   - 3.1 Types
   - 3.2 Values
   - 3.3 Identifiers
4. **Schema model**
   - 4.1 Schema
   - 4.2 Struct definition
   - 4.3 Enum and type alias definitions
   - 4.4 Type tables
   - 4.5 Meta-schema `core`
   - 4.6 Canonical form and hash
   - 4.7 Compiled schema files
5. **Primitive encodings**
   - 5.1 Variable-length unsigned integer (`vu`)
   - 5.2 Variable-length signed integer (`vs`)
   - 5.3 Fixed-width values
   - 5.4 Strings and deduplication
6. **File structure**
   - 6.1 Header
   - 6.2 Compression
   - 6.3 Schema section
   - 6.4 Body
7. **Value encoding**
   - 7.1 Scalars
   - 7.2 Nullable values
   - 7.3 Lists
   - 7.4 Maps
   - 7.5 Structs
   - 7.6 References (`ref<T>`)
   - 7.7 External references
   - 7.8 Polymorphism
   - 7.9 Any
8. **Writing**
   - 8.1 Determinism
   - 8.2 Streaming
9. **Schema evolution**
   - 9.1 Types
   - 9.2 Fields
   - 9.3 Type compatibility
10. **Resolution of external resources**
    - 10.1 Order
    - 10.2 Version matching
    - 10.3 Search paths
11. **Limits and errors**
12. **Example**

Appendix A. **Kind values**

Appendix B. **Conformance checklist for readers**

1. Introduction

SBDL (Sight Bundle) is a schema-driven serialization format. A file carries a header, one or more schemas (embedded, referenced, or omitted) and a single root value. Values are encoded **untagged**: the schema provides every type, so the data contains only payload bytes, except where the type cannot be known statically (polymorphism and `any`).

Design goals, in priority order:

1. Small files (variable-length integers, string deduplication, presence bitmaps, no field identifiers).
2. Fast single-pass reading and writing, with streaming support.
3. Schema evolution: data written with one version of a schema can be read with another.
4. Self-containment: a file can embed everything required to decode it.

This document defines the data model (§3), the schema model (§4), the binary encoding (§5–§8), schema evolution (§9), and resolution of external resources (§10). The text encoding is defined in the companion document; both encodings share §3, §4, §9 and §10.

1.1 Conventions

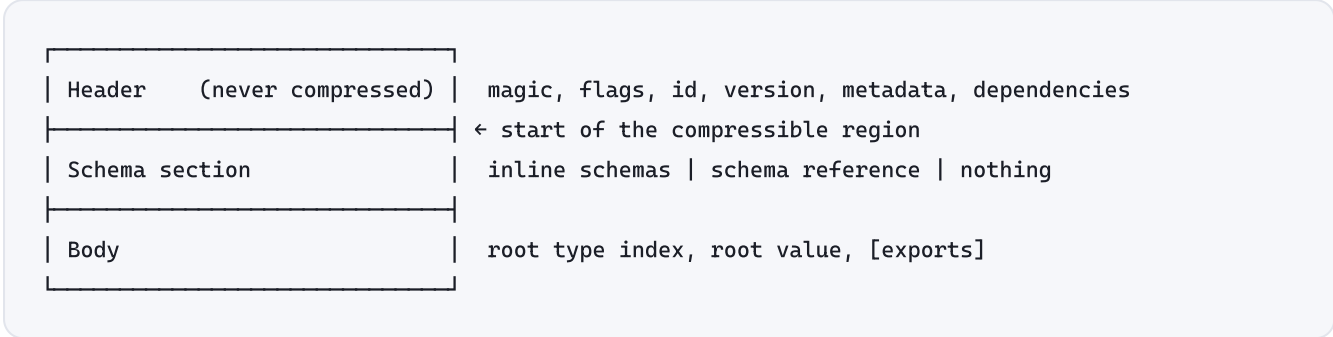
The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT** and **MAY** are to be interpreted as described in RFC 2119.

- All multi-byte fixed-width quantities are **little-endian**.
- A **reader** decodes files; a **writer** produces them. A file that violates a **MUST** of this document is **malformed**; a reader **MUST** reject it with an error and **MUST NOT** return a partial result.
- Byte values are written in hexadecimal (`0x2A`), bit 0 is the least significant bit.

1.2 Notation used in layouts

NOTATION	MEANING
`u8`	one raw byte
`u64le`	8 raw bytes, little-endian
`vu`	variable-length unsigned integer (§5.1)
`vs`	variable-length signed integer (§5.2)
`str`	string (§5.4)
`[x]`	`x` is present only under a condition stated alongside
`x × n`	`x` repeated `n` times

2. Overview



A single **string table** (§5.4) spans the schema section and the body. Header strings are never deduplicated.

3. Data model

3.1 Types

CATEGORY	TYPES
Boolean	`bool`
Unsigned integers	`u8`, `u16`, `u32`, `u64`
Signed integers	`s8`, `s16`, `s32`, `s64`
Floating point	`f32`, `f64` (IEEE 754 binary32 / binary64)
Text	`string` (Unicode, encoded as UTF-8)
User types	`enum`, `struct` (declared in a schema)
Containers	`list<T>`, `map<K, V>`
References	`ref<T>` (in place, internal or external)
Dynamic	`any`
Modifier	`T?` (nullable)

Constraints:

- `null` is a value, not a declarable type. It is the value of a nullable position, or an `any` value of kind `Null`.
- `T?` is allowed for every type except `any` (which can already hold `null`) and nullable types (`T??` is invalid).
- `K` in `map<K, V>` **MUST** be `bool`, an integer type, `string` or an `enum` type, and **MUST NOT** be nullable.
- `T` in `ref<T>` **MUST** be a struct type.

- A **type alias** (§4.3) gives a name to another type. It is not a distinct type: a position whose type is an alias has the type of its target, for every purpose (encoding, `any` values, schema evolution). `bytes`, in the text languages, is a built-in shorthand of `list<u8>`.

3.2 Values

- **Struct**: ordered set of named fields, as declared by its type and its base types.
- **Enum**: one of the named values of the enum type.
- **List**: ordered sequence of values of type `T`.
- **Map**: ordered sequence of `(key, value)` pairs with **unique** keys. Insertion order is preserved by the format.
- **Ref**: a struct value whose location is chosen value by value, in the data:
  - **in place**: the object is written at this position;
  - **internal**: the position points to an object of the same file. Several refs **MAY** point to the same object, and refs **MAY** form cycles. Identity is preserved by a round trip;
  - **external**: the position points to the root value or to a named export of another SBDL file (a *dependency*).
- **Any**: a value together with its type.

3.3 Identifiers

KIND	SYNTAX	EXAMPLE
Identifier	`[A-Za-z_][A-Za-z0-9_]*`	`Player`, `max_hp`
Qualified name	identifiers joined by `.`	`game`, `assets.textures`
Type identity	`<schema name>.<type name>`	`game.Player`

Field, enum value and type names are identifiers. Schema names are qualified names. File ids, dependency ids, metadata keys and export names are arbitrary non-empty strings, unless stated otherwise.

All name comparisons are ordinal (case-sensitive, code point by code point).

4. Schema model

4.1 Schema

A schema is a named, versioned set of type definitions:

PROPERTY	DESCRIPTION
`name`	qualified name, e.g. `game`
`version`	`u32`
`imports`	ordered list of `(name, version, hash)` of other schemas whose types are usable by this one

PROPERTY	DESCRIPTION
`types`	ordered list of type definitions (structs, enums and type aliases)

Rules:

- Type names **MUST** be unique within a schema.
- A schema **MUST NOT** import itself, directly or transitively.
- Only types of the schema itself and of its **direct** imports are usable in its definitions.
- A schema **MAY** also carry metadata (string keys and values). Metadata is not part of the schema model: it is not encoded in `core.Schema` and does not change the hash. It is stored in the header of compiled schema files (§4.7).

4.2 Struct definition

PROPERTY	DESCRIPTION
`name`	identifier
`base`	optional parent struct (single inheritance)
`isAbstract`	an abstract struct is never instantiated directly
`fields`	ordered list of fields
`isPositional`	text writers use the positional form `(a, b, c)` for values of this struct (text specification §6.6); no effect on the binary encoding

A **field** has a `name`, a `type`, an optional list of `aliases` (former names, §9) and an optional `default` value.

Rules:

- Inheritance **MUST NOT** be cyclic. A base **MUST** be a struct (not a type alias).
- Field names, including inherited ones, **MUST** be unique within a struct. An alias **MUST NOT** equal another field's name.
- A field **MAY** have a default only if its type is `bool`, an integer, a float, `string` or an enum, and is not nullable. The default **MUST** be a valid value of the field type.
- The **field sequence** of a struct is the fields of its root-most base first, then each derived level in order, ending with its own fields.
- A positional struct **MUST NOT** be abstract, and its field sequence **MUST** be non-empty and only contain fields whose type is `bool`, an integer, a float, `string` or an enum, not nullable (possibly through type aliases).

4.3 Enum and type alias definitions

Enum definition

PROPERTY	DESCRIPTION
`name`	identifier
`underlying`	one of `u8`, `u16`, `u32`, `u64`, `s8`, `s16`, `s32`, `s64`
`values`	ordered list of `(name, value)`

Value names **MUST** be unique, values **MUST** be unique and **MUST** fit the underlying type.

Type alias definition

PROPERTY	DESCRIPTION
`name`	identifier
`target`	aliased type, any type of §3.1 (including another alias)

Rules:

- The target **MUST NOT** be nullable nor fixed. A reference to an alias **MAY** add the nullable and fixed flags, under the rules of the resolved type (`Color?`, `Hash @fixed`).
- Aliases **MUST NOT** be cyclic, directly or through other aliases.
- An alias is resolved to its target wherever its type is used: values are encoded as the target (§7), `any` descriptors describe the target (§7.9), and schema evolution compares targets (§9.3). Renaming or removing an alias, or replacing a type by an alias of it, therefore never changes the data. It does change the schema hash.
- The target of `ref<T>` and the base of a struct **MUST** be struct definitions, not aliases.
- Data never designates an alias: no type index of the data (§7) refers to one.

4.4 Type tables

File schemas. A file uses an ordered, non-empty list of schemas, its **file schemas** (§6.3). Several schemas are needed when the data uses types of schemas that do not import each other, typically derived types declared in an extension schema (`mods.Dragon` deriving from `game.Character`: `game` cannot import `mods` without a cycle).

Schema closure. The closure of a list of schemas is the ordered list obtained by a depth-first, pre-order traversal of each schema of the list in order, following `imports` in declaration order, and skipping schemas already visited (compared by name). A closure **MUST NOT** contain two schemas with the same name and different versions.

File type table. The type table of a file is the concatenation of the `types` lists of every schema of the closure of the file schemas, in closure order. A **type index** in the data (§7) is an index in this table. Any struct or enum of the table may be used by the data, including as root type. Type aliases have an index too, but data never uses it.

Local type space. Inside a schema definition, type references use the **local** type space of that schema: its own `types`, followed by the `types` of each direct import in import order.

Concrete set. For a struct type `T`, the concrete set `C(T)` is `T` (if not abstract) plus every non-abstract struct deriving from `T` directly or indirectly that is present in the file type table, ordered by file type table index.

4.5 Meta-schema `core`

Schemas are themselves SBDL values of type `core.Schema`. The `core` schema, version 1, is built into every implementation and **MUST NOT** be redefined. It is defined as follows (in the schema language of the companion document):

```
@sgs 1
@id core
@version 1

enum Kind : u8 {                    // type index 0
  Null = 0, Bool = 1, U8 = 2, U16 = 3, U32 = 4, U64 = 5,
  S8 = 6, S16 = 7, S32 = 8, S64 = 9, F32 = 10, F64 = 11,
  String = 12, Enum = 13, List = 14, Map = 15, Struct = 16,
  Ref = 17, Any = 18, Alias = 19
}

struct SchemaRef {                  // 1
  name:    string
  version: u32
  hash:    u64 @fixed
}

struct TypeRef {                    // 2
  kind:  Kind
  flags: u8                         // bit 0 nullable, bit 1 fixed, other bits 0
  index: u32?                       // Enum, Struct, Ref, Alias: local type index
  args:  list<TypeRef>              // List: [T], Map: [K, V], otherwise empty
}

abstract struct TypeDef {           // 3
  name: string
}

struct StructDef : TypeDef {        // 4
  base:         u32?                // local type index of the parent
  isAbstract:   bool
  fields:       list<Field>
  isPositional: bool = false
}

struct EnumDef : TypeDef {          // 5
  underlying: Kind
  values:     list<EnumValue>
}
```

```
struct EnumValue {                  // 6
  name:  string
  value: s64
}

struct Field {                      // 7
  name:    string
  type:    TypeRef
  aliases: list<string>
  default: any                     // Null when the field has no default
}

struct Schema {                     // 8
  name:    string
  version: u32
  imports: list<SchemaRef>
  types:   list<TypeDef>
}

struct AliasDef : TypeDef {         // 9
  target: TypeRef
}
```

`C(TypeDef)` is `[StructDef, EnumDef, AliasDef]`: the polymorphism tags of type definitions are `0`, `1` and `2`.

`TypeRef` rules:

- `Ref` uses `index` for its target struct and has no `args`.
- `Enum` and `Struct` use `index`; primitives, `String` and `Any` have neither `index` nor `args`.
- `Alias` uses `index` for an `AliasDef` and has no `args`. Its `flags` are those of the reference, not of the target. `Alias` is only valid in a `TypeRef`, never in a type descriptor (§7.9).
- An `index` **MUST** designate a definition matching the kind: an `EnumDef` for `Enum`, a `StructDef` for `Struct` and `Ref`, an `AliasDef` for `Alias`.
- `Null` **MUST NOT** appear in a `TypeRef`.
- The **fixed** flag is only valid on `U16`, `U32`, `U64`, `S16`, `S32`, `S64` (for `Alias`, on the resolved kind). It changes the encoding (§7.1), not the type.
- `Field.default` is `Null` when the field has no default. Otherwise it **MUST** be a scalar `any` value of the field kind (e.g. `u16` for a `u16` field); for an enum field, it is the **name** of the enum value, as a `string` (enum values cannot be described with the `core` type table).

4.6 Canonical form and hash

The **canonical encoding** of a schema is the encoding of its `core.Schema` value (§7) with a fresh string table and string deduplication disabled (`dedupMax = 0`), without header.

The **hash** of a schema is XXH64 (seed 0) of its canonical encoding, written as `u64le`. Since `imports` contains the hashes of imported schemas, the hash covers the whole closure.

The hash of the built-in `core` schema is defined as `0`.

4.7 Compiled schema files

A compiled schema is an SBDL file whose:

- header `id` is the schema name, header `version` is the schema version, and header metadata is the schema metadata;
- schema section uses mode *Reference* to `core`, version 1, hash `0`;
- root value is the `core.Schema` value (type index 8).

A compiled schema contains a single schema; its imports are referenced by `SchemaRef` and resolved externally (§10).

5. Primitive encodings

5.1 Variable-length unsigned integer (`vu`)

The number `n` of leading `1` bits of the first byte (0 to 8) is the number of bytes that follow it. The remaining `7 − n` bits of the first byte (none for `n ≥ 7`) are the **high** bits of the value; the `n` following bytes are the **low** bits, little-endian:

```
value = LE(next n bytes) | (firstByte & (0x7F >> n)) << (8 × n)        for n ≤ 7
value = LE(next 8 bytes)                                                for n = 8
```

FIRST BYTE	TOTAL SIZE	VALUE BITS	MAXIMUM
`0xxxxxxx`	1	7	$2^{7} - 1$
`10xxxxxx`	2	14	$2^{14} - 1$
`110xxxxx`	3	21	$2^{21} - 1$
`1110xxxx`	4	28	$2^{28} - 1$
`11110xxx`	5	35	$2^{35} - 1$
`111110xx`	6	42	$2^{42} - 1$
`1111110x`	7	49	$2^{49} - 1$
`11111110`	8	56	$2^{56} - 1$
`11111111`	9	64	$2^{64} - 1$

Examples: `0` → `00`, `127` → `7F`, `128` → `80 80`, `300` → `81 2C`, `16384` → `C0 00 40`.

- Writers **MUST** use the shortest encoding. Readers **MUST** accept longer encodings.
- A decoded value outside the range of the target type is malformed.

5.2 Variable-length signed integer (`vs`)

ZigZag transform, then `vu`: `encoded = (v << 1) ^ (v >> 63)` on 64-bit two's complement. `0 → 0`, `-1 → 1`, `1 → 2`, `-2 → 3`…

5.3 Fixed-width values

TYPE	ENCODING
`bool`	`u8`: `0x00` false, `0x01` true; any other value is malformed
`u8`, `s8`	1 raw byte (two's complement for `s8`)
`f32`, `f64`	IEEE 754, 4 / 8 bytes little-endian; NaN payloads are preserved
fixed integers	2, 4 or 8 bytes little-endian (two's complement if signed)

5.4 Strings and deduplication

A string is a `vu x` followed, when inline, by bytes:

- `x & 1 = 0`: **inline** string of `len = x >> 1` bytes of UTF-8 that follow. If `0 < len ≤ dedupMax`, the string is appended to the string table.
- `x & 1 = 1`: string at index `x >> 1` of the string table. An index not yet assigned is malformed.

Rules:

- The string table is empty at the start of the schema section and is shared by the schema section and the body. `dedupMax` is read from the header (§6.1).
- Header strings (id, metadata, dependencies) are always inline and are **never** added to the string table, whatever their length. The dependency list is only known once the body is encoded (§8.2): keeping the header out of the table lets writers encode the body first and emit the header afterwards.
- Writers **MUST NOT** emit an inline string that is already present in the table (the table never contains duplicates).
- The bytes **MUST** be valid UTF-8 (no surrogate code points, no overlong forms).
- Length is in bytes. Strings longer than $2^{31} - 1$ bytes are malformed.

6. File structure

6.1 Header

FIELD	ENCODING	DESCRIPTION
magic	4 bytes	`53 42 44 4C` (`"SBDL"`)
format	`u8`	format version, `0x01`
flags	`u8`	see below
dedupMax	`u8`	maximum length (bytes) of deduplicated strings, `0` disables deduplication
id	`str`	file id, **MAY** be empty
version	`vu`	content version (`u32` range)
metadata	`[vu n, (str key, str value) × n]`	present if flag `M`; `n ≥ 1`, keys unique
dependencies	`[vu n, (str id, vu version) × n]`	present if flag `D`; `n ≥ 1`, ids non-empty and unique

Flags:

BITS	NAME	VALUES
0–1	schema mode	`0` Inline, `1` Reference, `2` None, `3` reserved
2	`M`	metadata present
3	`D`	dependencies present
4–5	compression	`0` none, `1` Deflate, `2`–`3` reserved
6	`E`	exports present
7	—	reserved, **MUST** be `0`

A reader **MUST** reject an unknown format version, a reserved value, or a reserved bit set.

The header is never compressed, so that it can be read without decompressing (used by search path indexing, §10.3).

6.2 Compression

If the compression field is `1`, every byte after the header, up to the end of the file, is a single raw Deflate stream (RFC 1951, no zlib or gzip wrapper). The decompressed bytes are decoded exactly as if they were stored

uncompressed. Bytes after the end of the Deflate stream are malformed.

6.3 Schema section

MODE	LAYOUT
Inline (`0`)	`vu n` (`n ≥ 1`), then `n` values of type `core.Schema` (§7). The list is the closure of the file schemas, in closure order (§4.4): it is itself the list of file schemas.
Reference (`1`)	`vu n` (`n ≥ 1`), then `n` × (`str name` , `vu version` , `u64le hash`): the file schemas, resolved externally (§10). Names **MUST** be unique.
None (`2`)	nothing — the reader **MUST** be given the file schemas by the caller.

Values of the schema section are decoded with the type table of the built-in `core` schema (§4.5), not with the file type table. They share the string table with the body.

Inline mode:

- The list **MUST** be closed (every import of a schema of the list is in the list) and in closure order, so that the file is self-contained and the type table is the same whether it is computed from the list or from its closure.
- Writers **MAY** prune types and imports that are unreachable from the values of the file. Pruned schemas keep their name and version; readers **MUST NOT** verify `SchemaRef.hash` between inline schemas.

Reference mode: readers **MUST** verify that the resolved schema has the given hash, except for the built-in `core` schema.

6.4 Body

FIELD	ENCODING
root type	`vu` — file type table index of a non-abstract struct
root value	value of that exact type (§7), without polymorphism tag
exports	`[vu n, (str name, vu typeIndex, ref value) × n]` — present if flag `E` ; `n ≥ 1`

Exports:

- `name` **MUST** be non-empty and unique within the file. It is the key used by external references of other files (§7.7).
- `typeIndex` is a non-abstract struct type. `ref value` is encoded as `ref<T>` (§7.6) where `T` is exactly that type: either a reference to an object already written, whose concrete type **MUST** equal `typeIndex` , or an inline object of that type, without polymorphism tag. An external reference (`x = 1`) is malformed here.

The file ends right after the body. Trailing bytes are malformed.

7. Value encoding

Values are encoded according to their **declared type** in the schema, with no type information unless stated otherwise.

7.1 Scalars

TYPE	ENCODING
`bool`	§5.3
`u8`, `s8`	§5.3
`u16`, `u32`, `u64`	`vu`; fixed: §5.3
`s16`, `s32`, `s64`	`vs`; fixed: §5.3
`f32`, `f64`	§5.3
`string`	§5.4
`enum`	the numeric value, encoded as the underlying type (`u8`/`s8` raw, others `vu`/`vs`). Values not declared by the enum are malformed.

7.2 Nullable values

- A nullable **struct field** is handled by the presence bitmap (§7.5).
- Any other nullable position (list element, map value, `TypeRef` argument) is prefixed by a `u8`: `0x00` null (nothing follows), `0x01` present (the value follows). Other values are malformed.

7.3 Lists

`vu count`, then the elements:

ELEMENT TYPE	ELEMENT ENCODING
`bool`	packed bits: `⌈count / 8⌉` bytes, element `i` is bit `i mod 8` of byte `i / 8`; unused bits **MUST** be 0
`u8`, `s8`, `f32`, `f64`, fixed integers (non-nullable)	contiguous raw block (`count × size` bytes)
other	each element encoded per its type

7.4 Maps

`vu count`, then `count` pairs `(key, value)`, each encoded per its type. Duplicate keys are malformed. Order is preserved.

7.5 Structs

```
[polymorphism tag]   vu        only if |C(T)| > 1, see §7.8
[presence bitmap]    ⌈b / 8⌉ bytes   only if b > 0
fields               each present field, in field sequence order
```

Presence bitmap. The `b` bitmap slots are the fields of the field sequence (§4.2) that are nullable or have a default, in that order. Slot `i` is bit `i mod 8` of byte `i / 8`; unused bits **MUST** be 0.

- Slot = 1: the field value follows, at its position in the sequence.
- Slot = 0: no bytes; the value is `null` (nullable field) or the field default **as declared in the writer schema**.

Writers **MUST** clear the slot of a field equal to its default. Equality is ordinal for strings, by numeric value for enums, and **bitwise** for floats.

Fields that are neither nullable nor defaulted are always written and have no slot.

7.6 References (`ref<T>`)

`vu x`, chosen by the writer for each value:

- `x = 0`: a new object follows inline, encoded as struct `T` (with polymorphism tag if needed). It receives the next object id. Ids start at 0 and are assigned in encoding order across the body (root, then exports), **before** the object's fields are encoded, so the object can reference itself or its ancestors.
- `x = 1`: external reference, followed by its target (§7.7).
- `x ≥ 2`: reference to the object of id `x − 2`, which **MUST** already exist; its concrete type **MUST** be `T` or derive from `T`.

The same object **MUST** be written inline once at most. A value written in place costs one byte more than the struct alone, and a reference to an object already written costs one `vu`.

7.7 External references

After `x = 1` (§7.6): `vu dependency`, then `str key`:

- `dependency` is an index in the header dependency list; out of range is malformed.
- `key = ""` designates the root value of the dependency, otherwise the export named `key`.

Readers **MAY** resolve the referenced value while decoding or later, on demand (§10). Its concrete type **MUST** be `T` or derive from `T`, compared by type identity (`schema.Type`), regardless of schema versions. A missing export or a type mismatch is an error at resolution time.

7.8 Polymorphism

For a position of declared struct type `T`:

- if `|C(T)| > 1`, a `vu` tag precedes the struct: the index of the concrete type in `C(T)`;
- otherwise no tag is written and the concrete type is the single element of `C(T)`. `C(T)` **MUST NOT** be empty for a type actually used in the data.

This applies to struct fields, list elements, map values, inline objects of `ref<T>`, and struct values in `any`. It does not apply to the root value and exports, whose exact type is already given.

7.9 Any

```
type descriptor
value             encoded per the descriptor (nothing for Null)
```

Type descriptor: one `u8` = kind (bits 0–4, `core.Kind` values), nullable (bit 5), fixed (bit 6), bit 7 = 0; followed by:

KIND	FOLLOWED BY
`Enum`, `Struct`, `Ref`	`vu` file type table index
`List`	descriptor of `T`
`Map`	descriptor of `K`, descriptor of `V`
others	nothing

Rules:

- The top-level descriptor of an `any` value **MUST NOT** have the nullable bit; a null value is kind `Null`.
- Kind `Alias` (19) is invalid in a descriptor: a value always carries its resolved type (§4.3).
- A struct described by the descriptor is the **exact** concrete type: no polymorphism tag follows.
- `Any` as element kind (e.g. `list<any>`) is allowed; each element then carries its own descriptor.

8. Writing

8.1 Determinism

For identical inputs (values, schema, options), writers **MUST** produce identical bytes. In particular: fields in field sequence order, map pairs and list elements in their order, strings registered in encoding order, and object ids assigned in traversal order.

8.2 Streaming

The dependency list is in the header but is only fully known once the body has been traversed (external references). Writers **MUST** therefore either:

- buffer the body (and schema section), then emit header + buffer; or
- write directly when every dependency is declared before writing starts; an external reference to an undeclared dependency is then an error.

Readers never need to seek: a file is decodable in a single forward pass.

9. Schema evolution

A reader decodes data with the **writer schema** (from the file) and binds it to a **reader schema** (expected by the caller). This binding is computed once per pair of schemas.

9.1 Types

Types are matched by type identity (`schema.Type`), schema versions excluded.

9.2 Fields

For each struct, a reader field `R` matches a writer field `W` when `W.name` equals `R.name` or one of `R.aliases`.

CASE	BEHAVIOR
`W` matches `R`	value decoded with `W` 's type, converted to `R` 's type (§9.3)
`W` has no match	value decoded and discarded
`R` has no match	`null` if nullable, default if defined, otherwise **error**

9.3 Type compatibility

Type aliases are resolved first, on both sides (§4.3). A writer type converts to a reader type if:

- they are equal; or
- integer widening: `u8 → u16 → u32 → u64` , `s8 → s16 → s32 → s64` , `uN → sM` with `M > N` ;
- `f32 → f64` ;
- `T → T?` ;
- enum → enum of the same identity: values matched **by name**; a writer value whose name is missing from the reader enum is an error at decoding time;
- struct → struct of the same identity, recursively per §9.2; a concrete writer type unknown to the reader is an error at decoding time;
- `list<A> → list<B>` , `map<K1, V1> → map<K2, V2>` , `ref<A> → ref<B>` when the arguments convert;
- anything → `any` .

Any other change is an error when computing the binding. The fixed flag never affects compatibility.

10. Resolution of external resources

Applies to schemas in Reference mode, imports in compiled schemas, and dependencies of external references. A resource is identified by `(id, version)` ; for schemas, `id` is the schema name.

10.1 Order

1. Resources provided in memory by the caller.
2. Custom resolvers, in registration order, returning a byte stream from `(id, version)` .

3. Search paths (§10.3).

The first source returning a result wins. Resources provided in memory are validated exactly like files (hash for schemas).

10.2 Version matching

The requested version **MUST** be found exactly. If the caller allows newer dependencies, and the exact version is not found, the highest available version greater than the requested one is used. Schemas always require an exact match and a matching hash.

10.3 Search paths

- Files with extension `.sbdl` (binary) and `.sbdt` (text) in each search path, including its sub-folders, are indexed by `(id, version)` by reading **their header only**. The index **MAY** be cached.
- Schema sources (`.sgs`, see the companion document) are also indexed, by their `@id` and `@version` directives, so that referenced schemas and imports can be resolved without compiling them first.
- The same `(id, version)` in several search paths: the first path in the list wins; shadowed files **SHOULD** be reported as diagnostics.
- The same `(id, version)` twice within the same search path is an error.
- Files with an empty id are not indexed.

11. Limits and errors

Readers **MUST** enforce configurable limits and fail with an error when exceeded. Recommended defaults:

LIMIT	DEFAULT
Nesting depth (structs, lists, maps, any)	128
Elements in a single list or map	2^{24}
String length	2^{24} bytes
Objects (`ref`) per file	2^{24}
Decompressed size	$2^{31} - 1$ bytes

Counts (lists, maps, metadata, …) whose minimal encoded size exceeds the remaining input **MUST** be rejected before allocation.

12. Example

Schema:

```
@sgs 1
@id demo
@version 1

struct Point {
  x:     s32
  y:     s32
  label: string?
}
```

Value: `Point { x: 3, y: -2, label: "p" }`, id `"p"`, version 0, inline schema, `dedupMax = 32`, no compression.

BYTES	MEANING
`53 42 44 4C`	magic `"SBDL"`
`01`	format version 1
`00`	flags: Inline schema, no metadata, dependencies, compression or exports
`20`	dedupMax = 32
`02 70`	id: inline, length 1, `"p"` (header strings are not deduplicated)
`00`	version 0
`01`	schema section: 1 schema (inline)
`08 64 65 6D 6F`	`Schema.name` : `"demo"` → string #0
`01`	`Schema.version` : 1
`00`	`Schema.imports` : empty
`01`	`Schema.types` : 1 element
`00`	polymorphism tag: `C(TypeDef)` = [StructDef, EnumDef, AliasDef] → StructDef
`00`	`StructDef` presence bitmap: `base` absent, `isPositional` equal to its default
`0A 50 6F 69 6E 74`	`name` : `"Point"` → string #1
`00`	`isAbstract` : false
`03`	`fields` : 3 elements

BYTES	MEANING
`02 78` `00 08 00 00` `00` `00`	field: `"x"`, TypeRef (bitmap, kind S32, flags 0, no args), no aliases, default `Null`
`02 79` `00 08 00 00` `00` `00`	field `"y"`, S32
`0A 6C 61 62 65 6C` `00 0C 01 00` `00` `00`	field `"label"`, String, flags = nullable
`00`	root type index 0 (`demo.Point`)
`01`	presence bitmap: `label` present
`06`	x = 3 (ZigZag 6)
`03`	y = −2 (ZigZag 3)
`02 70`	label: inline `"p"`

Total: 63 bytes, including the embedded schema.

Appendix A. Kind values

VALUE	KIND	VALUE	KIND
0	Null	10	F32
1	Bool	11	F64
2	U8	12	String
3	U16	13	Enum
4	U32	14	List
5	U64	15	Map
6	S8	16	Struct
7	S16	17	Ref
8	S32	18	Any
9	S64	19	Alias (`TypeRef` only)

Appendix B. Conformance checklist for readers

A reader **MUST** reject: wrong magic; unknown format version; reserved flag bits or values; invalid `bool` or presence byte; invalid UTF-8; string table index out of range; enum value not declared; polymorphism tag out of range; `Alias` kind in a type descriptor; cyclic type aliases; reference to an unassigned object id; dependency index out of range; duplicate map keys, metadata keys, dependency ids or export names; non-zero unused bits in bitmaps; schema hash mismatch (Reference mode); trailing bytes; exceeded limits.